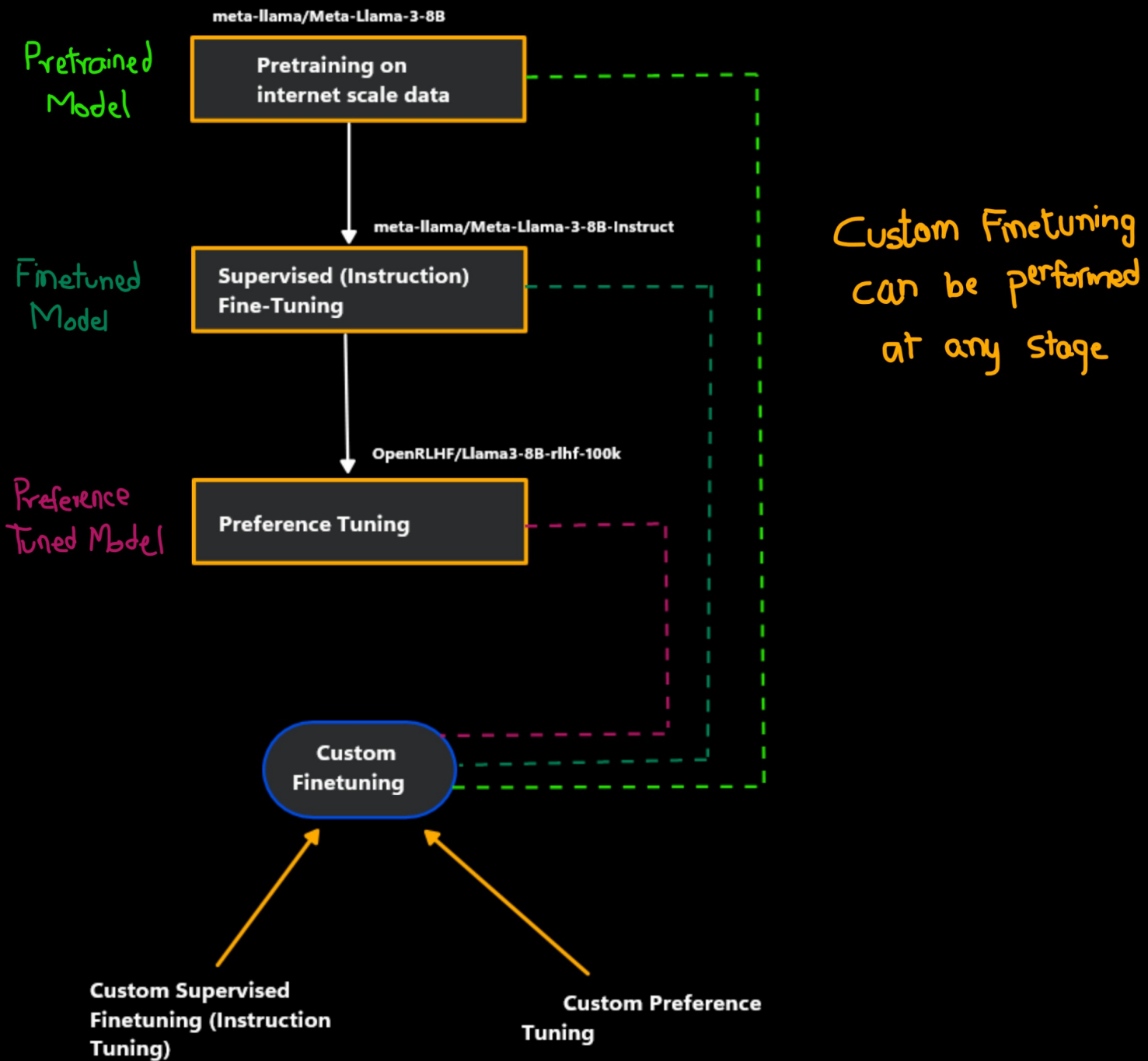


Fine-Tuning

Fine-tuning is the process of taking a already trained model and training it further on domain-specific or task-specific data so that it performs better for a particular use case

LLM Training Pipeline

- 1. Pretraining of a model
- 2. Supervised Finetuning (Instruction Finetuning)
- 3. Preference Tuning



Finetuning Frameworks

- 1. Huggingface Modules
- 2. Unsloth

Methods of Finetuning

- 1. Full finetuning
- 2. Parameter Efficient Finetuning

① Pretraining ← Raw

② SFT/Instruction ← I/P ↔ O/P

③ Preference Tuning

Custom
Tuning

Train the model
on your own
data

① Pretraining (Unsupervised/Selfsupervised Learning)

② SFT → Supervised Finetuning

Parameter Level

① Full Finetuning X → Retrain all parameters

② Partial Finetuning → Subset of Parameters

Two methods

① Old school method

CNN

BERT

BART

TS

① Freeze all the layers and
train only last output layer

② Freeze some starting layers and
retrain some remaining last layers

② PEFT

Small Infrastructure

① LORA/QLORA

(Weight Decomposition) ② DoRA

③ Bitfit

④ IA³

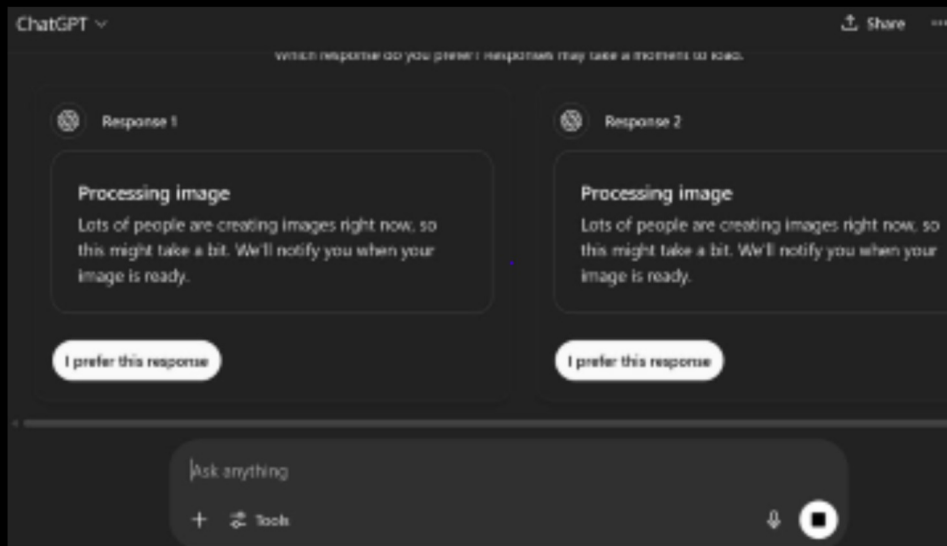
Quantized

float → int

↳ small float

X

③ Preference Alignment



① RLHF $\rightarrow$ OpenAI $\rightarrow$ RL $\rightarrow$ DRL $\rightarrow$ PPO

② GRPO

③ DPO

④ ORPO

Huggingface

Hugging Face is an AI company that provides a complete ecosystem for AI development

Mainly including training LLMs from scratch, fine-tuning, evaluation, hosting applications.

Almost the entire modern LLM fine-tuning industry workflow revolves around the Hugging Face ecosystem.

In modern Hugging Face ecosystem: **Most Important Libraries**

"If your goal is only modern LLM fine-tuning..."

Then these libraries are MUST KNOW:

- Transformers
- Datasets
- Tokenizers
- PEFT
- Accelerate
- TRL
- Bitsandbytes
- Safetensors
- Evaluate
- Lighteval

Evaluation Matrix:

- Accuracy
- F1-score
- BLEU
- ROUGE
- Perplexity
- Benchmark scores

Native vs Outsider Library

Not every tool is originally built by Hugging Face.

Native HF Libraries

- Transformers
- Datasets
- Tokenizers
- PEFT
- Accelerate
- TRL
- Diffusers
- Evaluate
- Optimum
- Safetensors

Outsider but Integrated

- Bitsandbytes
- timm
- Sentence Transformers
- Argilla
- Distilabel

So the ecosystem is integrated, even if ownership is different.

→ Datasets Library/ Custom Dataset

→ Preprocessing / Cleaning

→ Tokenizer (Tokenized Dataset)

→ Transformers Model Loading

→ PEFT (LORA Adapters)

→ Bitsandbytes (4-bit / Quantization)

→ Accelerate (Distributed & Multi-GPU Training)

→ Trainer / TRL (SFT, RLHF, DPO, PPO, GRPO)

→ Evaluation (Accuracy, BLEU, ROUGE, Perplexity, Benchmarks)

→ Save Checkpoints (Safetensors)

→ Push to Hugging Face Hub

→ Inference / Deployment (TGI, TEI, Inference Endpoints)

→ Monitoring & Iteration

Monitoring

- Accuracy / Quality Monitoring
- Latency Monitoring
- GPU / CPU Resource Monitoring
- Token Usage / Cost Monitoring

Iteration

- Add new training data
- Better LoRA adapters
- RLHF/DPO tuning
- Hyperparameter tuning

Unsloth

Unsloth is an ultra-optimized performance layer built on top of that foundation which enables:

- Faster training
- Lower VRAM usage
- Longer context training
- Cheaper fine-tuning

"Unsloth is an ultra-optimized LLM fine-tuning framework built on top of the Hugging Face ecosystem that enables fast LoRA/QLORA/RLHF training with very low VRAM usage."

Unsloth claims:

- 2x to 3x faster training
- 50–80% less VRAM usage
- Ability to train large models even on free GPUs"

Real Practical Meaning

- If a Hugging Face workflow takes:
 - 10 hours, 40 GB VRAM
- Then Unsloth may complete the same task in:
 - ~5 hours, 12–16 GB VRAM."

LLaMA Example

Take the example of LLaMA 3.1 8B:

- Standard HF: 24 GB VRAM
- Unsloth: only ~7–8 GB."

The most shocking feature of Unsloth is: LONG CONTEXT TRAINING.

20K–300K+ token training becomes possible.

"Where standard HF gives OOM...

Unsloth can handle extremely long contexts."

GPU vs Context Length: LLaMA 3.1 (8B)

12 GB → ~21K tokens

16 GB → ~40K

24 GB → ~78K tokens

80 GB → ~340K tokens

Whereas standard HF:

Even on an 80 GB GPU, only ~28K tokens are supported."

But HOW is Unsloth So Fast?

Now the question is...

How is this magic happening?

Answer:

Deep low-level GPU optimizations.

CUDA and Triton

CUDA is NVIDIA's low-level GPU programming layer.

It:

- runs on GPUs
- is massively parallel
- is extremely fast
- provides low-level control

Triton is a Python-like GPU language that generates highly optimized CUDA kernels."

Internal Optimizations

Internally, Unsloth uses:

- custom CUDA kernels
- Triton kernels
- fused attention
- optimized forward/backward pass
- smart checkpointing
- Flash Attention
- Manual backpropagation
- Automatic sequence packing

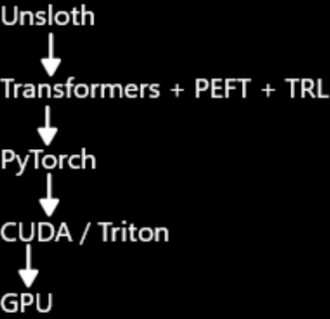
The most important point:
Unsloth does not rely on approximation tricks.
It maintains exact math.
That is why:

- Accuracy loss is negligible
- Model quality does not degrade"
- Architecture Flow

$2.3456 \times 8.9123 = 20.90518688$
 $2.3 \times 8.9 \approx 20.47$

Exact math means Unsloth improves speed and memory efficiency without changing the actual mathematical correctness of the model computations."

Internally, the stack looks something like this:"



This means Unsloth does not replace Hugging Face. Instead, it works like an optimized engine on top of Hugging Face."